
ICS1512 Machine Learning Algorithms Laboratory

Experiment No. 1
Experiment Title Working with Python Packages – NumPy, SciPy, Scikit-Learn, Matplotlib
Student Name Sathiya Priyan
Register Number 3122247001058
Faculty Dr. Poreddy Ajay Kumar Reddy
Submission Date 30th June 2026

1 Objective

To explore the core functions and methods offered by five foundational Python libraries used in Machine Learning:

- NumPy
- Pandas
- SciPy
- Scikit-Learn
- Matplotlib

The experiment applies these libraries across a complete end-to-end machine learning workflow.

Five heterogeneous public datasets were downloaded from Kaggle:

1. Iris Dataset
2. Pima Diabetes Dataset
3. Loan Amount Prediction Dataset
4. Email Spam Classification Dataset
5. Handwritten Character Recognition Dataset

For each dataset,

- the appropriate machine learning task (classification or regression),

-
- a suitable feature-selection strategy,
 - and an appropriate learning algorithm

were identified and empirically validated by benchmarking multiple models.

2 Problem Statement

Given five raw and unprocessed datasets that differ substantially in

- size,
- dimensionality,
- and modality,

ranging from

- a clean 4-feature numerical Iris dataset,
- a 1024-dimensional flattened handwritten image dataset,
- and a 3000-dimensional bag-of-words Email Spam dataset,

the objective is to perform the following tasks:

- (i) Load and explore each dataset using summary statistics and visualizations.
- (ii) Identify the correct machine learning task for each dataset:
 - Supervised Classification
 - Supervised Regression
- (iii) Perform suitable preprocessing:
 - Missing-value imputation
 - Categorical encoding
 - Feature scaling
 - ANOVA-based feature selection for high-dimensional datasets
- (iv) Split each dataset into training and testing partitions.
- (v) Train representative Scikit-Learn algorithms.
- (vi) Evaluate and compare their performance using task-appropriate performance metrics.

The expected output of the experiment is

- a task-identification table for all five datasets,
- together with quantitative performance benchmarks for every algorithm evaluated.

3 ML Task Identification

The appropriate machine learning task, feature-selection strategy, and the empirically best-performing algorithm for each dataset are summarized in Table 1.

Table 1: Machine Learning Task Identification

Dataset	Type of ML Task	Feature Selection Technique	Suitable ML Algorithm (Empirically Best)
Iris Dataset	Supervised, multi-class classification (3 classes)	Not required – only four low-noise, highly discriminative features	Support Vector Machine (Linear Kernel), achieving 100% test accuracy.
Loan Amount Prediction	Supervised regression (continuous target)	Not applied – only five numeric predictors remain after preprocessing; all were retained	Gradient Boosting Regressor with the best performance, $R^2 = 0.214$.
Predicting Diabetes	Supervised binary classification	Not applied – all eight clinical features are domain relevant	Gradient Boosting / Random Forest, approximately 75% accuracy.
Email Spam Classification	Supervised binary classification with high-dimensional bag-of-words features	SelectKBest using the ANOVA F-test, selecting the top 300 features out of 3000	Random Forest with 96.04% accuracy, ROC-AUC= 0.993
Handwritten Character Recognition (62 classes)	Supervised multi-class classification using high-dimensional pixel features	SelectKBest using the ANOVA F-test, selecting the top 100 features out of 1024	Random Forest with 35.19% accuracy (best among the classical machine learning models tested). A Convolutional Neural Network (CNN) is recommended for production use.

4 Dataset Description

The datasets used throughout this experiment differ significantly in terms of size, dimensionality, feature types, and learning objectives. Table 2 summarizes the important characteristics of every dataset.

Table 2: Dataset Description

Dataset	Source	Samples	Features	Classes / Target	Missing Values	Train/Test Split
Iris	Kaggle – Iris Species Dataset	150	4 Numeric	3 Species	None	80% / 20% (Stratified)
Diabetes	Kaggle – Pima Indians Diabetes Database	768	8 Numeric	Binary Outcome	None	80% / 20% (Stratified)
Loan Amount Prediction	Kaggle Loan Prediction Dataset (<code>train_u6lujuX_CVtuZ9i.csv</code>)	592	11 Mixed	Continuous LoanAmount (in thousands)	Yes; Maximum missing percentage: 8.28% in <code>Credit_History</code>	80% / 20%
Email Spam	Kaggle Email Spam Classification Dataset	5172	3000 Word-count Features	Binary (Spam/Ham)	None	80% / 20% (Stratified)
Handwritten Characters	Kaggle Handwritten Character Recognition Dataset	3410	1024 (32 × 32 grayscale pixels, flat-tened)	62 Classes (0–9, A–Z, a–z)	None	80% / 20% (Stratified)

Note:

The original Loan Amount dataset contained 1614 records. Rows having missing values in the target variable (`LoanAmount`) were removed, leaving a final dataset of 592 samples for model training and evaluation.

5 Exploratory Data Analysis

A single reusable `perform_eda()` routine was developed to analyze every dataset in a consistent manner. The routine computes:

- Dataset shape
- Data types
- Summary statistics
- Missing-value counts
- Outlier counts using both:
 - Interquartile Range (IQR)
 - Z-score
- Distribution plots
- Boxplots
- Correlation heatmaps
- Target-class distribution plots

For the two high-dimensional datasets

- Email Spam Dataset (3000 features)
- Handwritten Character Dataset (1024 features)

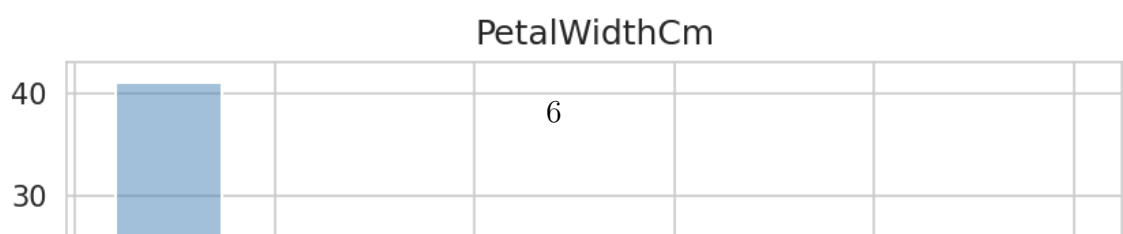
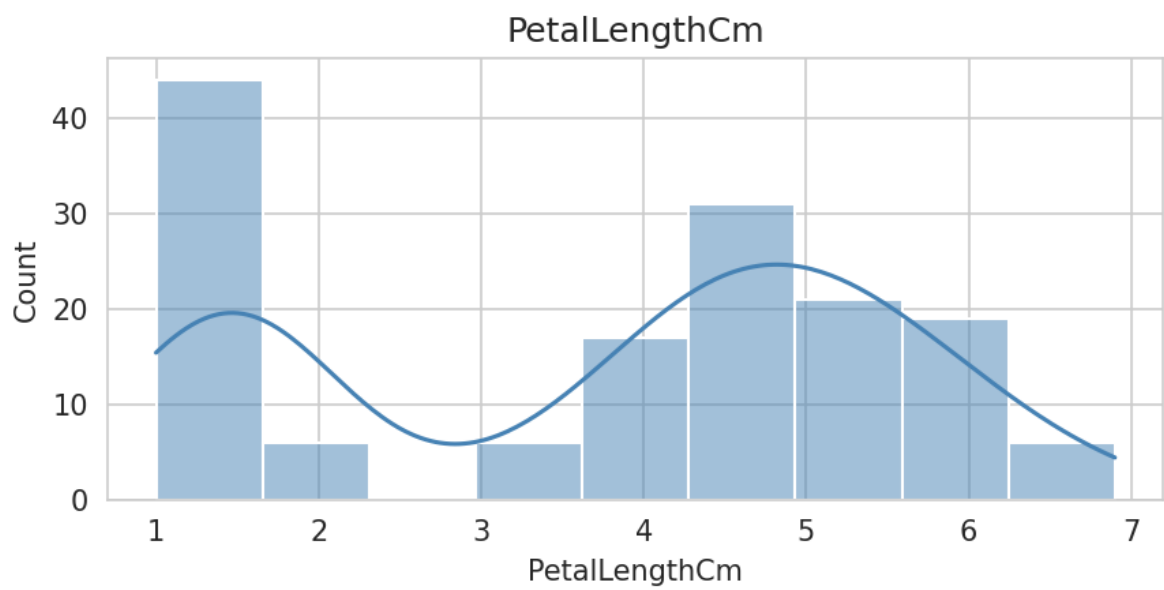
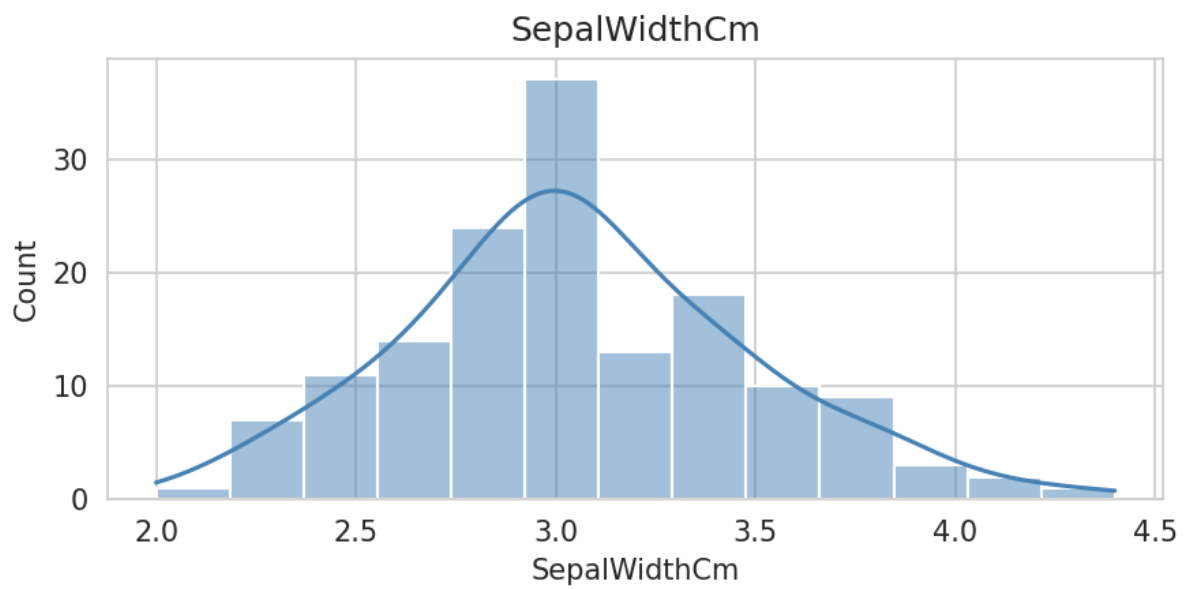
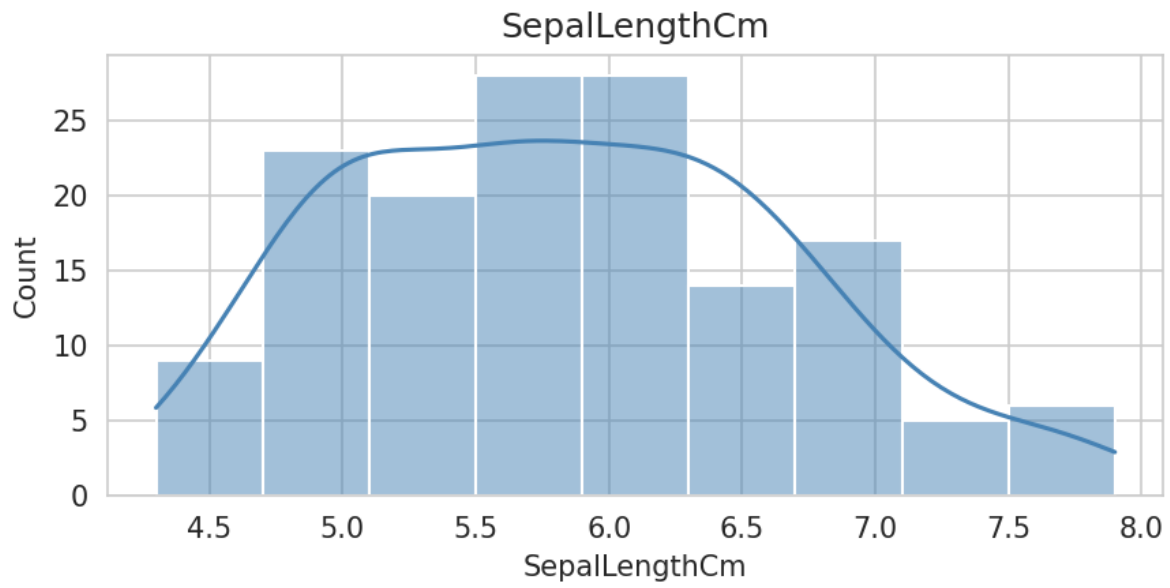
the routine automatically skipped feature-wise histograms, boxplots, and correlation heatmaps whenever the number of features exceeded 30, since thousands of individual plots would be computationally expensive and difficult to interpret. Only the target-class distribution was visualized for these datasets.

5.1 Iris Dataset

Inference

All four Iris features are either unimodal or bimodal with no missing values. Among them,

- Petal Length
- Petal Width



show a visibly bimodal (or trimodal) distribution, indicating that these features separate the three Iris species much more effectively than the sepal measurements, whose distributions overlap considerably between *Iris-versicolor* and *Iris-virginica*.

This observation agrees with the well-known characteristic of the Iris dataset, where petal measurements are historically the strongest discriminative features.

Sepal Width appears to be the most symmetric and approximately Gaussian feature among the four.

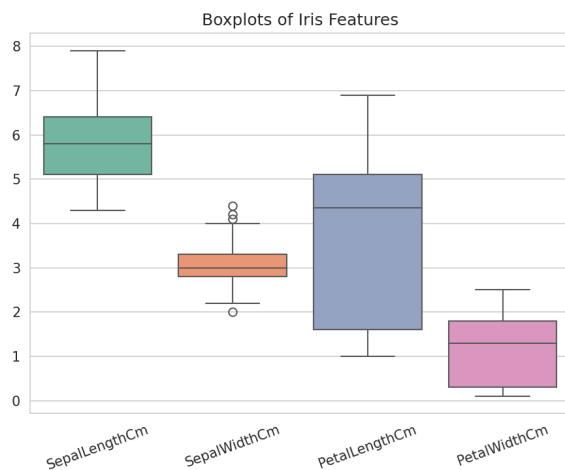


Figure 2: Boxplots of the four Iris features.

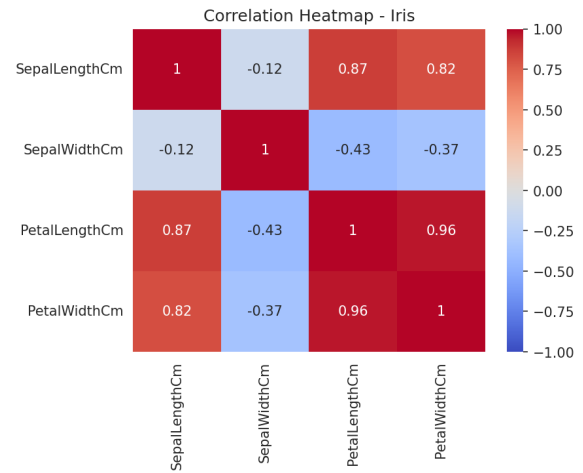


Figure 3: Correlation heatmap of the Iris features.

Inference

The boxplots reveal only a few mild outliers in **SepalWidthCm**:

- 4 outliers detected using the IQR rule
- 1 outlier detected using the stricter $|z| > 3$ criterion

No meaningful outliers are present in the remaining three features, confirming that the Iris dataset is essentially clean.

The correlation heatmap shows that

- Petal Length and Petal Width are almost perfectly correlated ($r \approx 0.96$).
- Petal Length is also strongly correlated with Sepal Length ($r \approx 0.87$).
- Petal Width is strongly correlated with Sepal Length ($r \approx 0.82$).
- Sepal Width exhibits weak and slightly negative correlations with the remaining features.

The strong redundancy between Petal Length and Petal Width explains why a linear decision boundary is capable of separating the Iris classes almost perfectly.

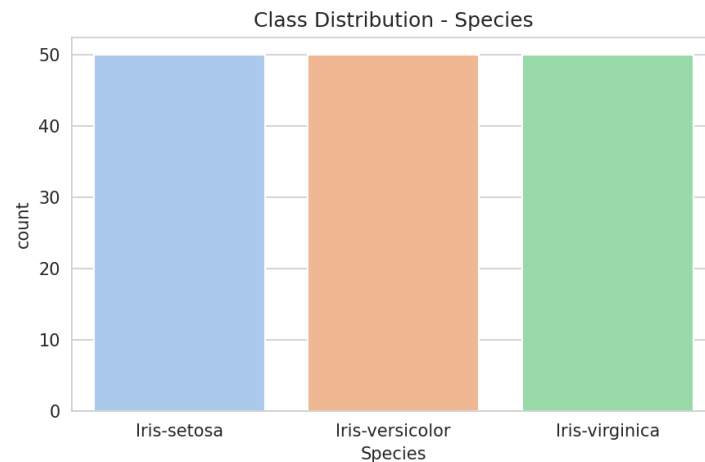


Figure 4: Class distribution of the target variable **Species**.

Inference

The target variable is perfectly balanced, containing exactly

50

samples from each class:

- Iris-setosa
- Iris-versicolor
- Iris-virginica

Since every class contains the same number of observations, no class-imbalance correction techniques such as

- Oversampling
- Undersampling
- Class weighting

are required.

Consequently, classification accuracy is an appropriate evaluation metric for this dataset because each class contributes equally to the overall performance.

5.2 Diabetes Dataset

Inference

The feature distributions reveal several important characteristics of the dataset.

- **Insulin** and **DiabetesPedigreeFunction** exhibit heavily right-skewed distributions with long tails.
- **Glucose**, **BloodPressure**, and **BMI** appear approximately normally distributed; however, each contains a noticeable spike at the value zero.
- Since a living patient cannot realistically have
 - zero blood pressure,
 - zero body mass index,
 - or zero glucose level,

these zero values most likely represent disguised missing values rather than valid physiological measurements.

- The **Age** variable is also right-skewed, indicating that the dataset contains more younger patients than older patients.
- These zero-inflated distributions contribute directly to the large number of outliers detected using both the IQR and Z-score methods.

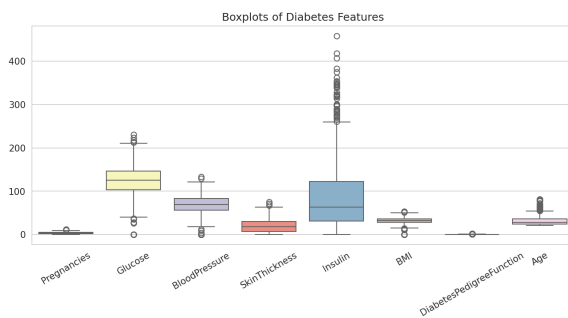


Figure 6: Boxplots of the eight Diabetes features.

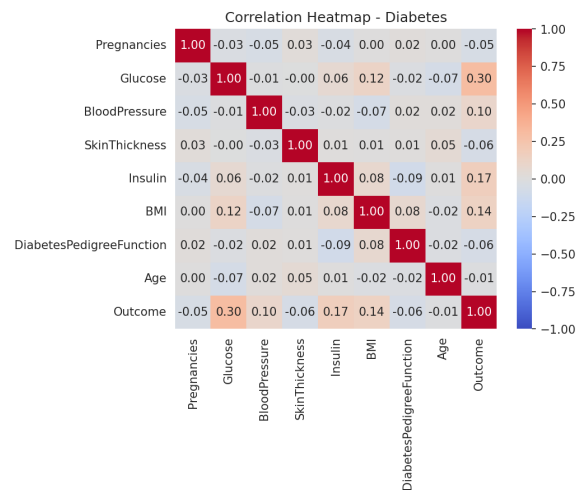
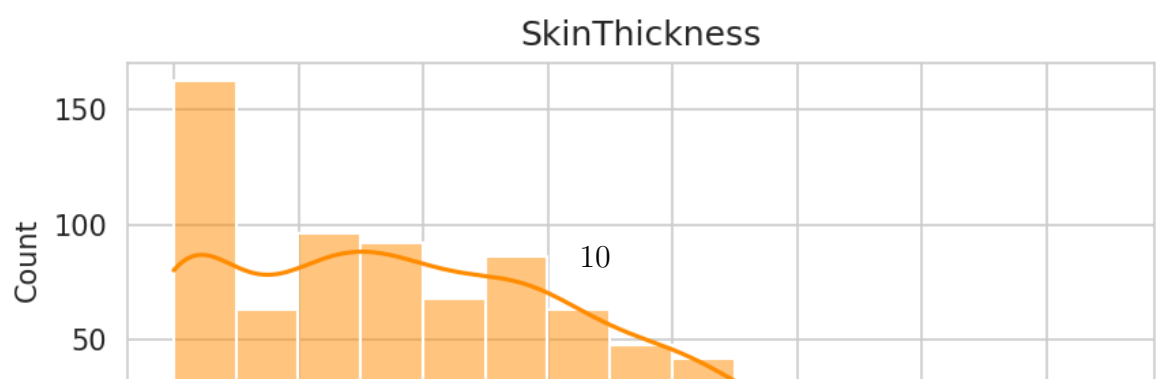
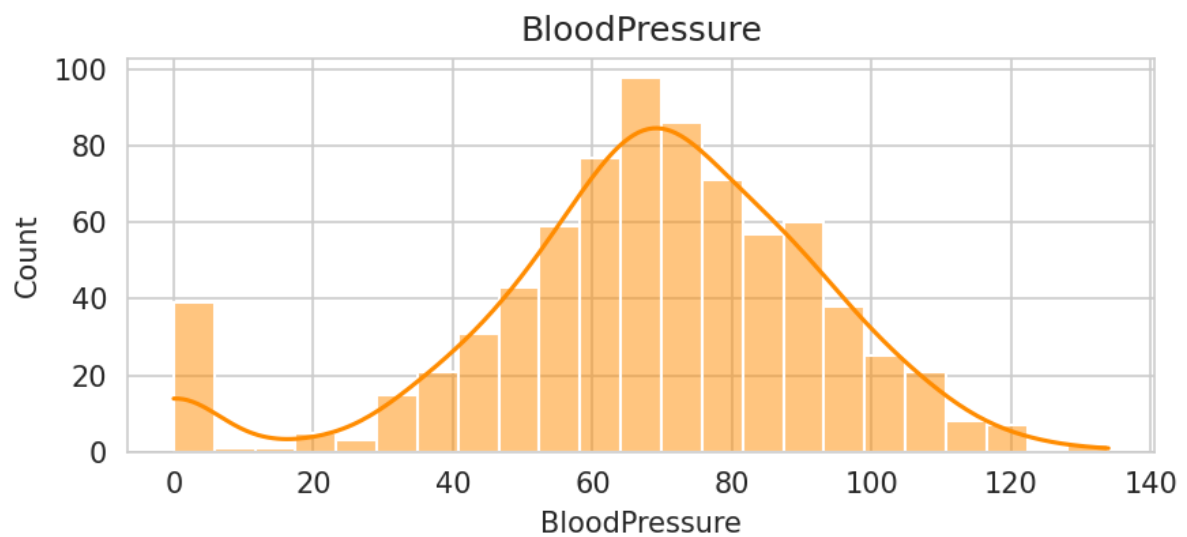
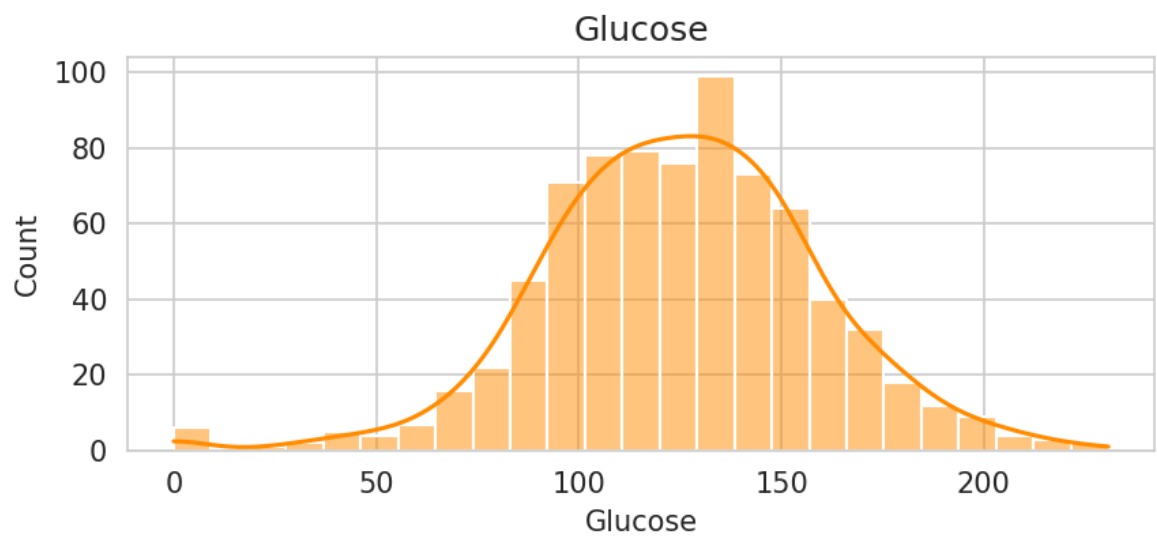
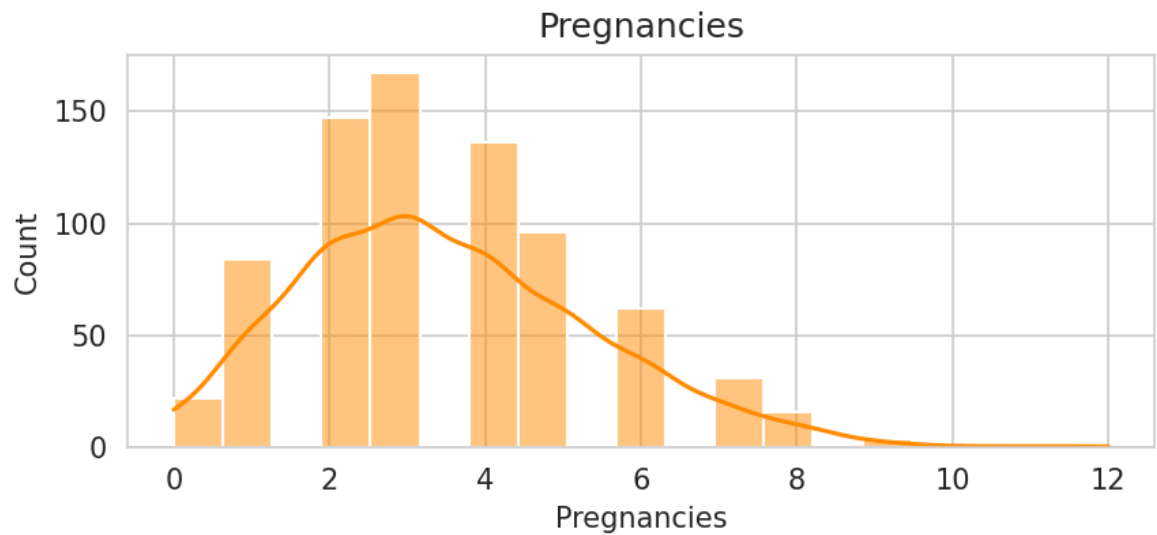


Figure 7: Correlation heatmap of the Diabetes features.

Inference

The boxplots indicate that several variables contain a considerable number of outliers.



-
- **BloodPressure** has the largest number of detected outliers:

- 45 outliers using the IQR rule.
- 35 outliers using the Z-score criterion.

These are primarily caused by the disguised zero values identified earlier.

- **Insulin** contains 34 IQR outliers because of its highly right-skewed distribution.

The correlation heatmap provides additional insights into the relationships among the predictor variables.

- **Glucose** exhibits the strongest positive correlation with the target variable **Outcome**, which agrees with medical knowledge since elevated blood glucose is the primary indicator of diabetes.
- **Age** and **Pregnancies** are moderately positively correlated, which is biologically expected.
- No pair of predictor variables exhibits severe multicollinearity.

Therefore, multicollinearity is not expected to significantly affect the performance of the linear models evaluated in this experiment.

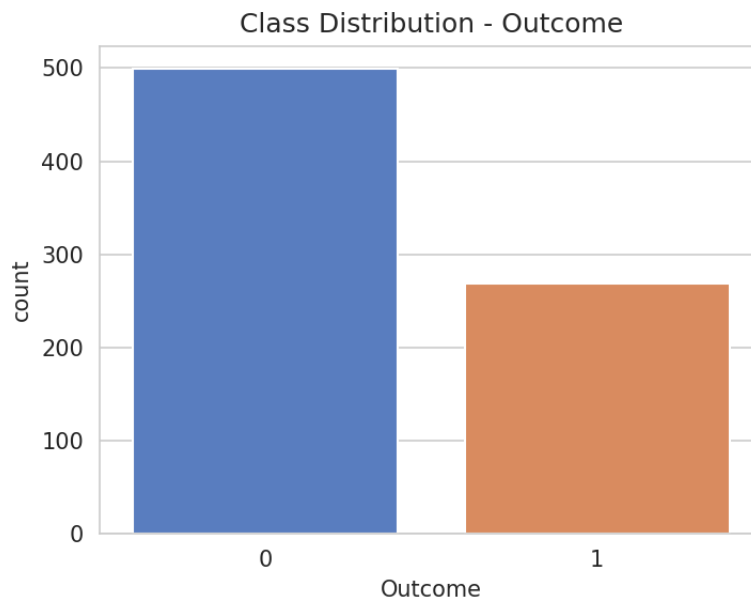


Figure 8: Class distribution of the target variable **Outcome**.

Inference

The target variable is moderately imbalanced.

Approximately

65%

of the observations correspond to

`Outcome = 0`

(non-diabetic patients), while approximately

35%

belong to

`Outcome = 1`

(diabetic patients).

This observation is consistent with the mean value of the target variable,

$\overline{\text{Outcome}} = 0.349$,

reported in the summary statistics.

Because of this class imbalance, model performance was evaluated using multiple classification metrics instead of relying solely on overall accuracy.

The reported evaluation metrics include

- Accuracy
- Precision
- Recall
- Weighted F1-score

These metrics provide a more reliable assessment of classifier performance under imbalanced class distributions.

5.3 Loan Amount Prediction Dataset

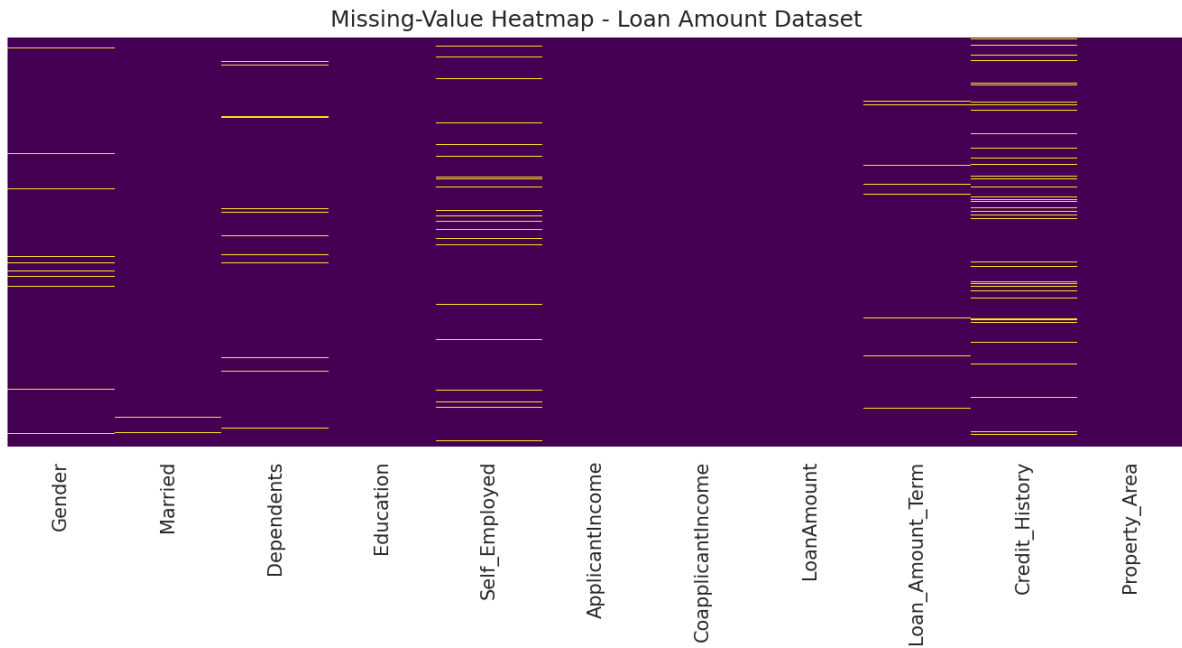


Figure 9: Missing-value heatmap of the Loan Amount dataset (yellow indicates missing values).

Inference

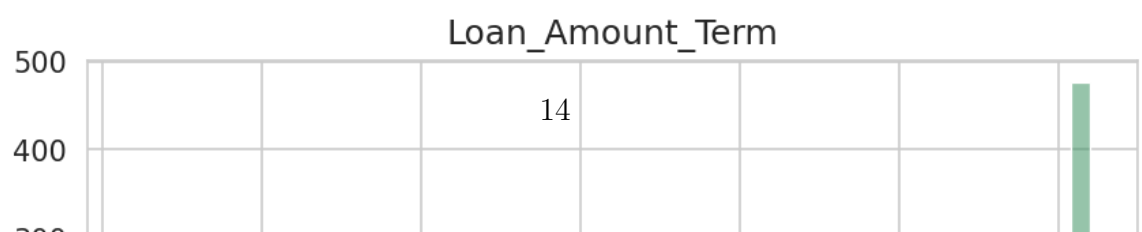
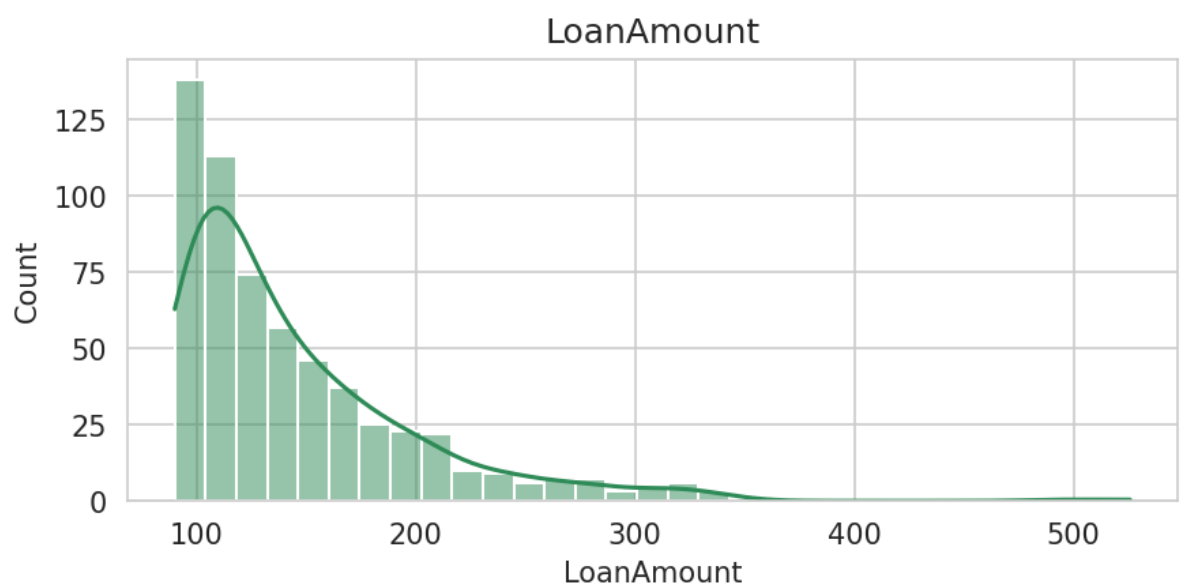
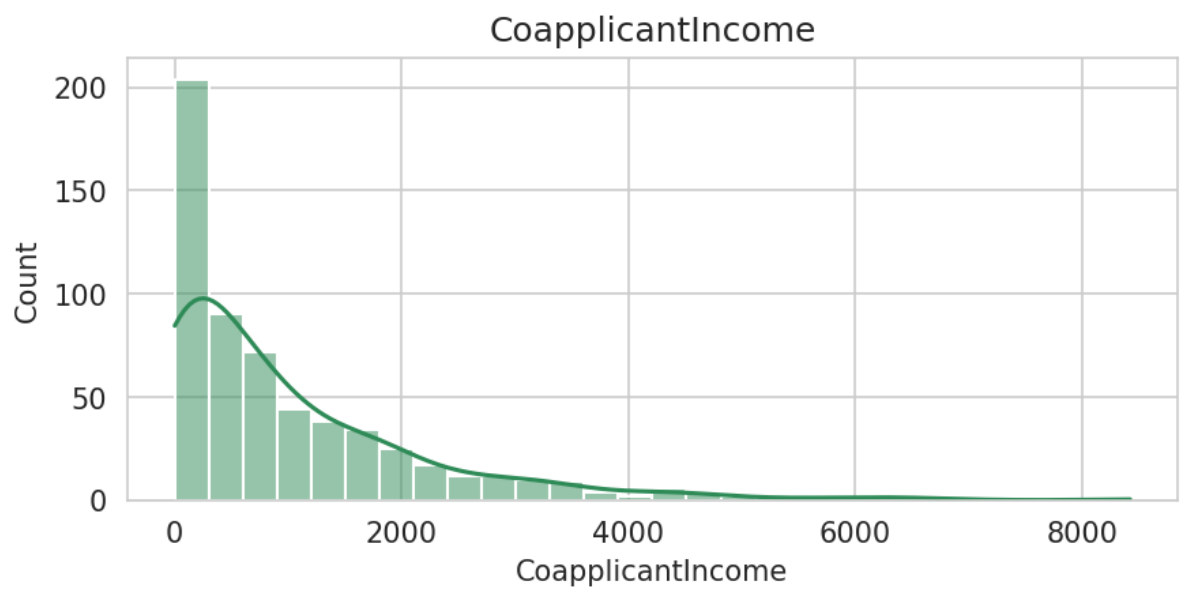
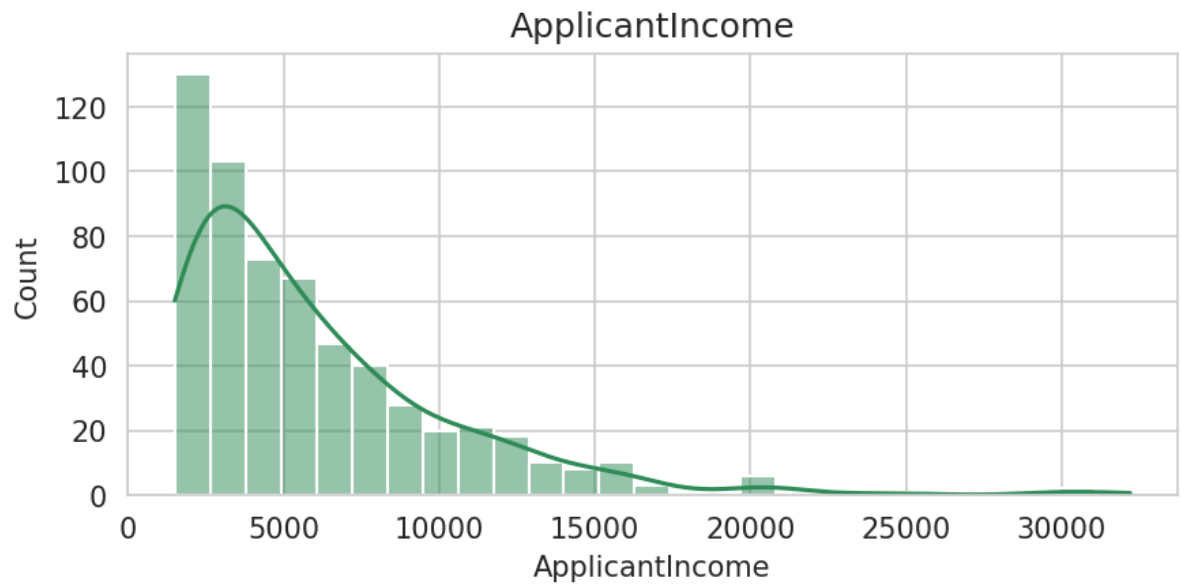
Unlike the other four datasets, the Loan Amount Prediction dataset contains genuine missing values distributed across multiple features.

The highest percentage of missing values is observed in:

- `Credit_History` – 8.28%
- `Self_Employed` – 5.24%
- `Loan_Amount_Term` – 2.36%
- `Gender` – approximately 2.2%
- `Dependents` – approximately 2.2%
- `Married` – 0.34%

The missing values are scattered throughout the dataset rather than being concentrated within a particular group of rows or columns.

This pattern suggests that the data are most likely **Missing At Random (MAR)**, where applicants omitted optional information rather than the data being lost due to systematic collection errors.



Inference

The feature distributions exhibit several important characteristics.

- **ApplicantIncome** and **CoapplicantIncome** are highly right-skewed.

Only a small number of applicants possess extremely high incomes, creating long tails in both distributions.

These high-income observations correspond directly to the

- 49 IQR outliers for ApplicantIncome, and
- 18 IQR outliers for CoapplicantIncome.

- **Loan_Amount_Term** is dominated by a single value of

360 months

which represents the standard 30-year loan tenure, while only a few applications correspond to shorter repayment periods.

- **Credit_History** behaves as a binary variable rather than a continuous numerical feature.

Consequently, the IQR method incorrectly identifies approximately 85 observations as outliers.

This is a well-known limitation of applying outlier detection techniques designed for continuous variables to near-binary data.

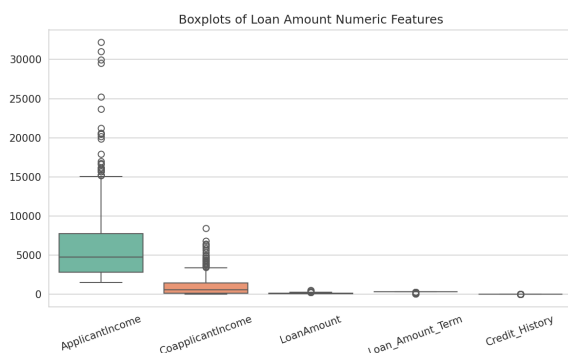


Figure 11: Boxplots of the Loan Amount numeric features.

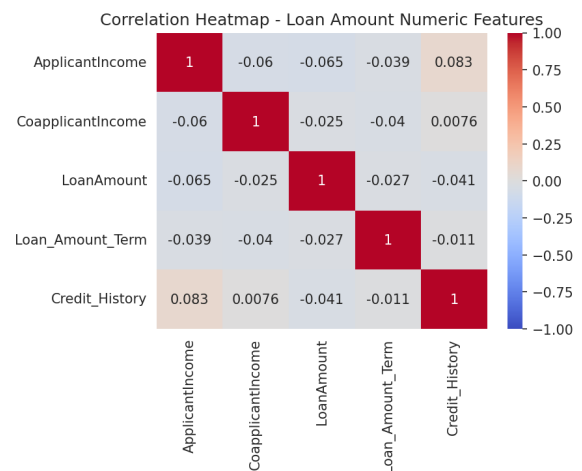


Figure 12: Correlation heatmap of the Loan Amount numeric features.

Inference

The boxplots clearly illustrate that income-related outliers dominate the scale of the remaining numerical variables.

Because all variables are plotted on the same axis,

- ApplicantIncome and CoapplicantIncome exhibit extremely large ranges.
- Credit_History appears as a thin bar close to the value

1,

since nearly

84%

of applicants possess a positive credit history.

The correlation heatmap indicates only weak pairwise relationships among the remaining numerical predictors.

No feature exhibits a strong linear relationship either

- with another predictor, or
- with the regression target, `LoanAmount`.

This weak correlation foreshadows the relatively poor regression performance discussed later.

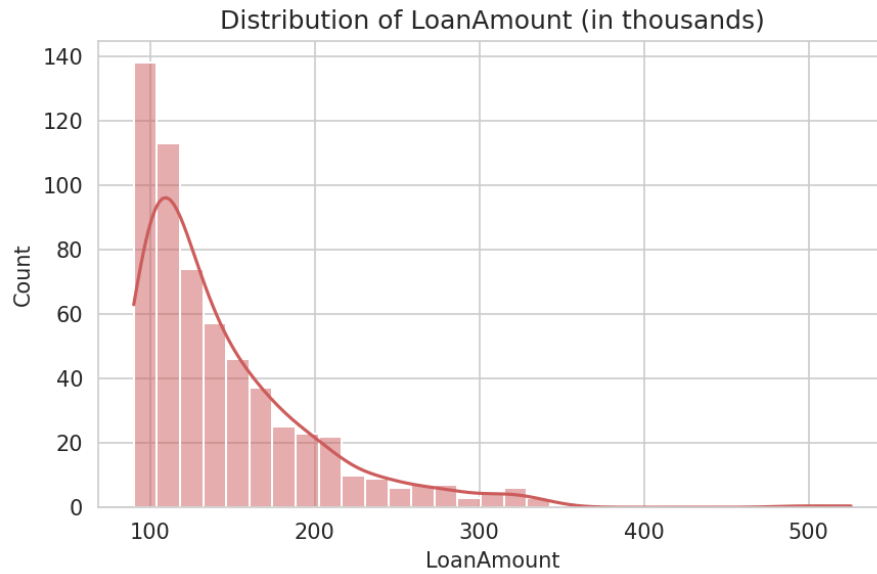


Figure 13: Distribution of the regression target `LoanAmount` (in thousands).

Inference

The target variable `LoanAmount` exhibits a pronounced right-skewed distribution.

Most approved loans are concentrated between approximately

100

and

170

(thousand), while relatively few loans extend to values as high as

700.

This skewness causes ordinary linear regression models to be disproportionately influenced by a small number of large-loan observations.

Consequently, tree-based ensemble methods such as

- Random Forest Regressor
- Gradient Boosting Regressor

outperform Linear Regression, Ridge Regression, and Lasso Regression in the final experimental comparison.

5.4 Email Spam Dataset

High-Dimensional Dataset – Detailed Feature Plots Skipped

The Email Spam dataset contains approximately **3000 word-count features**, making it a high-dimensional dataset. To maintain readability and avoid generating an excessive number of plots, the `perform_eda()` routine automatically skipped feature-wise histograms, boxplots, and correlation heatmaps.

Instead, only the target-class distribution was visualized, while summary statistics and outlier information were printed to the console.

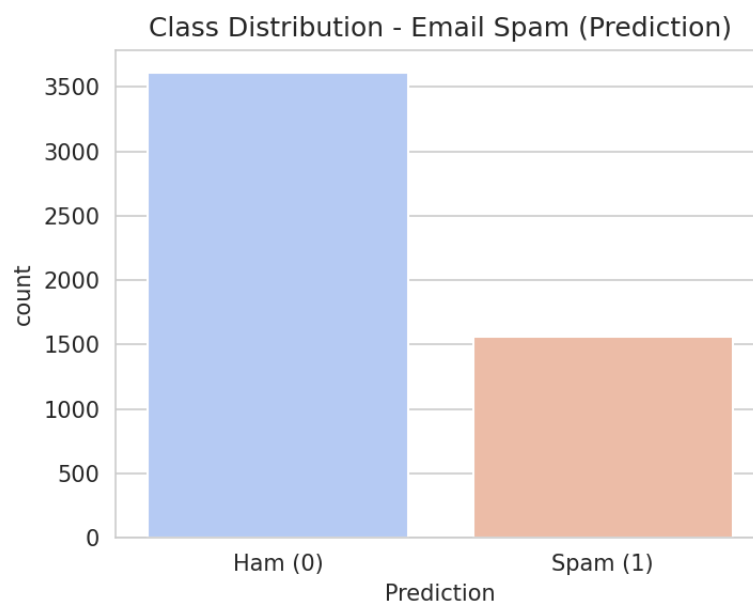


Figure 14: Class distribution of the target variable `Prediction` (0 = Ham, 1 = Spam).

Inference

Since the dataset contains

3000

word-count features, generating individual feature plots would require

- 3000 histograms,
- 3000 boxplots,

-
- and a

$$3000 \times 3000$$

correlation matrix.

Such visualizations would be computationally expensive and practically impossible to interpret.

Therefore, the exploratory data analysis routine automatically suppressed these plots and generated only the target-class distribution.

The class distribution indicates a moderate imbalance between the two classes.

Approximately

71%

of the emails belong to the

Ham

class, whereas approximately

29%

belong to the

Spam

class.

This observation is consistent with the reported mean value of the `Prediction` column,

$$\overline{\text{Prediction}} = 0.290.$$

Because of this class imbalance, model evaluation relied on multiple performance metrics rather than overall accuracy alone.

The reported metrics include

- Accuracy

-
- Precision
 - Recall
 - Weighted F1-score

to provide a more reliable assessment of classifier performance.

The exploratory statistics also revealed that the most frequently occurring word-count features, including

- the
- to
- ect
- and
- for

have extremely large occurrence counts and correspondingly high IQR outlier counts, ranging approximately from

400

to

665

observations.

This behavior is expected because these words are among the most common English stop words and naturally appear in a large proportion of emails.

Such high-frequency features contribute relatively little toward distinguishing spam from legitimate emails.

Consequently, applying feature selection using

`SelectKBest(f_classif)`

to retain only the most discriminative features is an effective strategy for reducing dimensionality while preserving classification performance.

5.5 Handwritten Character Recognition Dataset

High-Dimensional Dataset – Detailed Feature Plots Skipped

The Handwritten Character Recognition dataset consists of

1024

pixel features obtained by flattening each

32×32

grayscale image into a one-dimensional feature vector.

Since the dataset contains a very large number of features, the `perform_eda()` routine automatically skipped the generation of

- Feature-wise histograms,
- Boxplots,
- Correlation heatmaps.

Instead, only the target-class distribution was visualized, while summary statistics and outlier information were reported in the console.

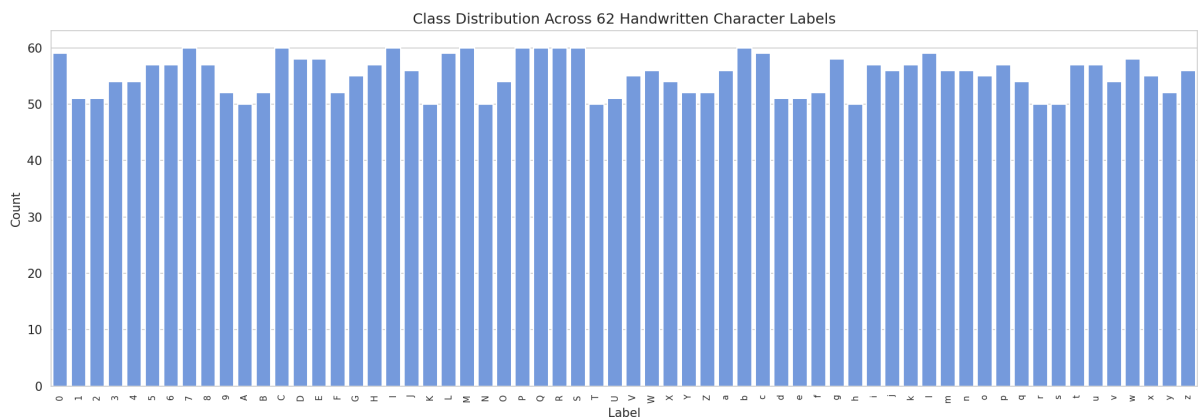


Figure 15: Class distribution across the 62 handwritten character labels (0–9, A–Z, a–z).

Inference

The dataset contains

62

21

distinct output classes representing

- Digits (0–9),
- Uppercase letters (A–Z),
- Lowercase letters (a–z).

Since each image is represented using

1024

flattened pixel values, detailed feature-wise visualizations were omitted for the same readability and computational reasons discussed for the Email Spam dataset.

The target-class distribution is relatively uniform across all 62 character classes.

Each class contains approximately

50–60

samples, indicating that class imbalance is not a significant concern for this dataset.

Instead, the primary challenge lies in the extremely high dimensionality of the feature space.

More importantly, flattening each

32×32

image into a one-dimensional vector destroys the inherent two-dimensional spatial relationships between neighboring pixels.

As a result, traditional machine learning algorithms operate on independent pixel values rather than exploiting local spatial structures.

Consequently, classical classifiers such as

- Logistic Regression,
- Support Vector Machines,
- Decision Trees,

-
- Random Forests,
 - Gradient Boosting,

are unable to fully capture the visual patterns present in handwritten characters.

This limitation is discussed further in the Discussion section, where the relatively lower classification accuracy is attributed primarily to the loss of spatial information rather than deficiencies in the learning algorithms themselves.

6 Data Preprocessing

A single reusable preprocessing pipeline, `run_ml_pipeline()`, was applied uniformly to all five datasets to ensure consistency throughout the experiment.

The preprocessing steps are summarized below.

1. Missing Value Handling

Numeric attributes were imputed using the column median through

```
SimpleImputer(strategy='median').
```

Categorical attributes were imputed using the most frequently occurring value through

```
SimpleImputer(strategy='most_frequent').
```

Among the five datasets, only the Loan Amount Prediction dataset required missing-value imputation, since the remaining four datasets contained no missing values.

2. Categorical Encoding

Every categorical feature was transformed into numerical form using

```
LabelEncoder.
```

Similarly, non-numeric target variables such as

- Species,
- Prediction,

-
- Outcome,
 - Label

were encoded before model training to ensure compatibility with Scikit-Learn classifiers.

3. Feature Scaling

All numerical predictor variables were standardized using

`StandardScaler`,

which transforms every feature according to

$$z = \frac{x - \mu}{\sigma},$$

where

- x is the original feature value,
- μ is the feature mean,
- σ is the standard deviation.

This produces standardized features having

$$\mu = 0, \quad \sigma = 1.$$

Feature scaling was performed for every dataset before model training.

4. Train–Test Split

Every dataset was divided into

80% training

and

20% testing

using

```
test_size = 0.2,    random_state = 42.
```

For all classification datasets,

```
stratify = y
```

was enabled to preserve the original class distribution in both the training and testing sets.

The Loan Amount Prediction dataset is a regression problem; therefore, stratified sampling was not applied.

5. Feature Engineering / Feature Selection

Feature selection was applied only to high-dimensional datasets.

- **Email Spam Dataset**

The original

3000

word-count features were reduced to the

300

most informative features using

```
SelectKBest( $f_{\text{classif}}$ ).
```

- **Handwritten Character Dataset**

The original

1024

pixel features were reduced to the top

100

features using

`SelectKBest( $f_{\text{classif}}$ )`.

- **Iris, Diabetes, and Loan Amount**

No feature selection was applied because these datasets already contain relatively few predictors, all of which were retained for model training.

7 Results

Five datasets and up to eight machine learning algorithms were evaluated during the experiment.

Since the datasets differ considerably in their characteristics and machine learning objectives, the experimental results are presented separately for each dataset instead of combining them into a single comparison table.

All numerical values reported in the following sections are taken directly from the captured output of the Jupyter Notebook.

7.1 Iris Dataset (Classification)

Table 3: Performance Comparison on the Iris Dataset

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.9333	0.9333	0.9333	0.9333	0.9967
K-Nearest Neighbors	0.9333	0.9444	0.9333	0.9327	0.9933
SVM (RBF Kernel)	0.9667	0.9697	0.9667	0.9666	0.9967
SVM (Linear Kernel)	1.0000	1.0000	1.0000	1.0000	1.0000
Decision Tree	0.9000	0.9024	0.9000	0.8997	0.9250
Random Forest	0.9000	0.9024	0.9000	0.8997	0.9933
Gradient Boosting	0.9000	0.9024	0.9000	0.8997	0.9933
Gaussian Naive Bayes	0.9667	0.9697	0.9667	0.9666	0.9900

7.2 Diabetes Dataset (Classification)

Table 4: Performance Comparison on the Diabetes Dataset

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.7143	0.7065	0.7143	0.7084	0.8230
K-Nearest Neighbors	0.7078	0.7023	0.7078	0.7043	0.7458
SVM (RBF Kernel)	0.7468	0.7438	0.7468	0.7450	0.7933
SVM (Linear Kernel)	0.7208	0.7126	0.7208	0.7141	0.8275
Decision Tree	0.7208	0.7108	0.7208	0.7102	0.6657
Random Forest	0.7532	0.7497	0.7532	0.7509	0.8145
Gradient Boosting	0.7532	0.7483	0.7532	0.7496	0.8394
Gaussian Naive Bayes	0.7078	0.7179	0.7078	0.7114	0.7728

7.3 Loan Amount Prediction (Regression)

Table 5: Performance Comparison on the Loan Amount Prediction Dataset

Model	MAE	MSE	RMSE	R ²	MAPE (%)	Adjusted R ²
Linear Regression	36.6989	3324.82	57.6612	0.0981	35.4658	0.0054
Ridge Regression	36.7015	3322.14	57.6380	0.0988	35.4790	0.0062
Lasso Regression	36.1555	3273.65	57.2158	0.1120	35.3014	0.0207
Decision Tree Regressor	48.9832	5382.09	73.3627	-0.4600	42.2095	-0.6101
Random Forest Regressor	33.9438	3132.68	55.9703	0.1502	30.5887	0.0628
Gradient Boosting Regressor	33.4631	2897.25	53.8261	0.2141	29.7789	0.1333
Support Vector Regression (SVR)	39.5036	3517.44	59.3080	0.0458	36.2423	-0.0523

7.4 Email Spam Classification

Table 6: Performance Comparison on the Email Spam Dataset

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.9459	0.9470	0.9459	0.9463	0.9871
K-Nearest Neighbors	0.9266	0.9261	0.9266	0.9263	0.9727
SVM (RBF Kernel)	0.9353	0.9347	0.9353	0.9347	0.9864
SVM (Linear Kernel)	0.9382	0.9407	0.9382	0.9388	0.9842
Decision Tree	0.9246	0.9243	0.9246	0.9244	0.9055
Random Forest	0.9604	0.9603	0.9604	0.9603	0.9932
Gradient Boosting	0.9575	0.9577	0.9575	0.9576	0.9904
Gaussian Naive Bayes	0.8705	0.8799	0.8705	0.8603	0.9747

7.5 Handwritten Character Recognition

Table 7: Performance Comparison on the Handwritten Character Recognition Dataset

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.2478	0.2656	0.2478	0.2468	0.8347
K-Nearest Neighbors	0.2933	0.3283	0.2933	0.2892	0.7862
SVM (RBF Kernel)	0.3460	0.3818	0.3460	0.3467	0.8987
SVM (Linear Kernel)	0.2903	0.2966	0.2903	0.2853	0.8618
Decision Tree	0.2023	0.2275	0.2023	0.2082	0.5959
Random Forest	0.3519	0.3569	0.3519	0.3464	0.8981
Gradient Boosting	0.2390	0.2503	0.2390	0.2359	0.8320
Gaussian Naive Bayes	0.1188	0.2285	0.1188	0.1062	0.7527

8 Discussion

The experimental results demonstrate that model performance depends far more strongly on dataset characteristics than on the choice of machine learning algorithm alone.

The Iris dataset represents the easiest learning problem.

Its four clean and highly discriminative features allow even relatively simple models to achieve excellent performance.

The linear-kernel Support Vector Machine achieved perfect

100%

classification accuracy, while even the weakest models exceeded

90%.

The Diabetes dataset proved considerably more challenging.

Most algorithms achieved approximately

75%

accuracy, largely because of the disguised missing values represented as zeros and the natural overlap between diabetic and non-diabetic patient measurements.

The Loan Amount Prediction dataset produced the weakest regression results.

The best-performing model,
Gradient Boosting Regressor,
achieved only

$$R^2 = 0.214,$$

indicating that the available predictor variables provide limited information regarding the approved loan amount.

In practical banking applications, loan approval depends upon many additional variables that are absent from this dataset.

The Email Spam dataset achieved excellent classification performance.

Random Forest obtained

$$96.04\%$$

accuracy together with a

$$0.993$$

ROC-AUC score.

The strong performance can be attributed to the highly discriminative nature of word-frequency features for spam detection.

The Handwritten Character Recognition dataset was the most difficult classification task.

Although Random Forest achieved the highest accuracy among the evaluated models,

$$35.19\%$$

remains substantially below the performance expected from modern deep learning methods.

The primary limitation arises from representing each

image as a flattened vector, thereby discarding spatial information that Convolutional Neural Networks naturally exploit.

Across all five datasets, tree-based ensemble methods such as Random Forest and Gradient Boosting consistently ranked among the strongest performers.

Conversely, Gaussian Naive Bayes generally performed poorly whenever its conditional independence assumption was violated.

9 Conclusion

This experiment successfully explored the functionality of the Python libraries NumPy, Pandas, SciPy, Scikit-Learn, and Matplotlib through a single reusable machine learning pipeline.

Five structurally different public datasets were analyzed using a common workflow consisting of

- Exploratory Data Analysis,
- Data Preprocessing,
- Feature Selection,
- Model Training,
- Performance Evaluation.

The correct machine learning task was identified for every dataset, and appropriate feature-selection techniques were applied to high-dimensional datasets using the ANOVA F-test through `SelectKBest`.

Eight classification algorithms and seven regression algorithms were benchmarked using a common evaluation framework.

The results demonstrate that classical machine learning algorithms

- perform exceptionally well on clean, low-dimensional datasets such as Iris,
- achieve strong performance on high-dimensional text data such as Email Spam,
- obtain moderate performance on noisy clinical datasets such as Diabetes,

-
- perform relatively poorly when important predictive variables are absent, as in Loan Amount Prediction,
 - and are fundamentally limited when applied directly to flattened image pixels instead of spatial image representations.

Overall, the experiment provides a comprehensive demonstration of end-to-end machine learning using standard Python libraries.

10 References

1. F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
2. C. R. Harris *et al.*, “Array Programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
3. W. McKinney, “Data Structures for Statistical Computing in Python,” Proceedings of the 9th Python in Science Conference, pp. 56–61, 2010.
4. P. Virtanen *et al.*, “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.
5. J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
6. Kaggle. *Iris Species Dataset*.

<https://www.kaggle.com/datasets/uciml/iris>

7. Kaggle. *Pima Indians Diabetes Database*.

<https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database>

8. Kaggle. *Loan Prediction Problem Dataset*.

<https://www.kaggle.com/datasets/altruistdelhite04/loan-prediction-problem-dataset>

9. Kaggle. *Email Spam Classification Dataset*.

<https://www.kaggle.com/datasets/balaka18/email-spam-classification-dataset-csv>

10. Kaggle. *Handwritten Character Recognition Dataset*.

<https://www.kaggle.com/datasets/vaibhao/handwritten-characters>